SLICE 13 RECON — on-hand stock enforcement + post-sale reset

Recon only. No code was changed in this slice. Every claim below cites the file and line it was read from on 2026-09-03 against `main @ d50e2151`. Nothing here is recalled or inferred; where I could not prove something I say so.

Two defects reported by the owner after counter testing:

- **Bug A** — the register lets you add unlimited quantity of a product regardless of what is actually in stock. (Legal per-category limits DO work and are correctly enforced — confirmed below. This is only about on-hand.)
- **Bug B** — after a completed sale the register locks as designed, but entering the PIN reopens the SAME customer and the SAME cart, skipping the age gate.

PART 1 — BUG A: on-hand quantity is never enforced

1.1 The stock number already exists and already reaches the register

This is the good news: no new plumbing is required.

Fact	Anchor
Server publishes the variant count into the menu bundle	`src/app/api/pos/menu/route.ts:174` — `unitsLeft: hasRealVariants ? variant.inventoryLevel : null`
The register's product type carries it	`src/lib/pos/sale-flow-core.ts:92` — `unitsLeft?: number | null`
Its documented meaning	`sale-flow-core.ts:86-91` — "*variant-level units remaining from the published menu, when known. null = unknown ... Warnings only — the B19 decrement + completion gate are the authority.*"

That comment at `:86-91` is the design decision that produced this bug. It was a deliberate choice, not an oversight — but it is the wrong choice for the owner's requirement, and changing it is what Slice 14 must do.

1.2 The two functions that actually mutate the cart — neither looks at stock

`src/lib/pos/sale-flow-core.ts:201-211`:

```
export function addToCart(cart, product) {
  const existing = cart.find((e) => e.product.variantId === product.variantId);
  if (existing) {
    return cart.map((e) =>
      e.product.variantId === product.variantId
        ? { ...e, quantity: Math.min(MAX_LINE_QUANTITY, e.quantity + 1) }   // :206
        : e,
    );
  }
  return [...cart, { product, quantity: 1 }];                               // :210
}
```

`src/lib/pos/sale-flow-core.ts:214-218`:

```
export function setCartQuantity(cart, variantId, quantity) {
  const q = Math.min(MAX_LINE_QUANTITY, Math.floor(quantity));              // :215
  if (q <= 0) return cart.filter((e) => e.product.variantId !== variantId);
  return cart.map((e) => (e.product.variantId === variantId ? { ...e, quantity: q } : e));
}
```

The only ceiling in either function is `MAX_LINE_QUANTITY = 99` (`sale-flow-core.ts:198`). `product.unitsLeft` is in scope at `:202` — the whole product object is right there — and is never read. That is Bug A in two lines.

Proof the 99 cap is the ONLY cap, from the existing self-test at `sale-flow-core.ts:560`: `ok(setCartQuantity(cart, "var-flower-35", 500)[0].quantity === MAX_LINE_QUANTITY, "quantity capped")` — asking for 500 units yields 99, not "however many are in stock".

1.3 Every path into the cart routes through those two functions

I traced all of them so the fix cannot miss one:

#	Path	Anchor
1	Tile tap (browse overlay)	`SaleFlow.tsx:2401` → `setCart(addToCart(cart, p))`
2	Tile tap (main grid)	`SaleFlow.tsx:3100` → `setCart(addToCart(cart, p))`
3	`manualAdd()` helper	`SaleFlow.tsx:2104` → `setCart(addToCart(cart, p))`
4	Enter-key barcode in search box (`tryScan`)	`SaleFlow.tsx:2130`
5	Global wedge scan (`handleGlobalScan`)	`SaleFlow.tsx:2155`
6	`+` / `−` steppers on a cart line	`SaleFlow.tsx:2530` and `:2532` → `setCartQuantity(...)`
7	Trash / remove	`SaleFlow.tsx:2542` → `setCartQuantity(cart, variantId, 0)`

All seven funnel through `addToCart` / `setCartQuantity`. This is the single most important finding for the fix plan: enforcing inside those two pure functions covers every entry point at once, including the Socket scanner work coming in Slice 12, which will land on path 5.

1.4 The overage is ALREADY CALCULATED — and then only whispered

`src/lib/pos/low-stock-core.ts:78-88` already does the exact arithmetic:

```
if (typeof line.unitsLeft !== "number" || !Number.isInteger(line.unitsLeft) || line.unitsLeft < 0) { ... }  // :78
if (line.quantity > line.unitsLeft) {                                                                        // :82
  `${label}: cart has ${line.quantity}, menu shows only ${line.unitsLeft} left — check the shelf before promising.`  // :84
}
```

That string is produced, carried into `SaleFlow.tsx:2231-2243` as `stockWarnings`, and displayed. The register **already knows** it is overselling and says so quietly. The comment at `SaleFlow.tsx:2229-2230` states the policy outright: "*Advisory only; the sale is never blocked on a cached number.*"

So Bug A is not a missing calculation. **It is a missing consequence.**

1.5 The tender gate omits stock — the precedent to copy

`SaleFlow.tsx:2272-2273`:

```
const canTender =
  cart.length > 0 && priced.problems.length === 0 && !limits.blocked && highThcViolations.length === 0;
```

Four conditions. Legal category limits (`limits.blocked`), statutory price floors (`priced.problems`), and the High-THC medical rule (`highThcViolations`) all hard-block the Tender button (`:2876` `disabled={!canTender}`). **Stock is absent from this expression.** This line is exactly why Michael sees legal limits enforced but not stock — his observation was precisely right, and this is the line that proves it.

`limits.blocked` is the working pattern to mirror: computed at `SaleFlow.tsx:2224-2227`, rendered as an OVER/NEAR/OK meter at `:2663-2670`, explained at `:2701`, and wired into `canTender`.

1.6 What the server does today — the honest picture

Stock does **not** silently corrupt to a negative number in the database. `src/lib/inventory/sale-decrement-core.ts:42-43` documents: "*OVERSELL: levels are clamped at 0 (never stored negative …) and every clamp is reported*", implemented at `:212-217` (`if (after < 0) { ...oversold.push(...) } workingLevel.set(variant.rowId, Math.max(0, after))`) and self-tested at `:386-388`.

So the real-world damage is not a negative integer in a column. It is:

1. A customer is **promised product the store does not have**, at the counter.
2. The oversell note at `:214` says "*cycle count to reconcile*" — someone has to clean it up later.
3. The published menu and the shelf drift apart, which is the exact Cultivera pain described in `held-stock-core.ts:7-12`.

I want to be precise: I have **not** yet traced whether that oversell report is surfaced anywhere the owner actually sees. That is listed as an open question in §4 rather than asserted either way.

1.7 The honest limitation the fix must respect

`unitsLeft` comes from the **cached published menu bundle**, not a live query (`menu/route.ts:174`). Between menu refreshes it can lag the physical shelf. That is the true reason the original author chose advisory-only, and the fix must not pretend the number is perfect:

- `unitsLeft === null` or `undefined` means **unknown**, and must never block. Confirmed: custom sales set `unitsLeft: null` (`custom-sale-core.ts:146`) and website-order carts do too (`order-to-cart-core.ts:134`). If we blocked on unknown, we would break custom sales entirely.
- A negative or non-integer value is corrupt and must be ignored, matching the existing guards at `low-stock-core.ts:78` and `held-stock-core.ts:88`.
- Therefore the enforcement is: **block only when we have a trustworthy non-negative integer AND the requested quantity exceeds it.**
- A manager override must exist, because the cached number CAN be wrong and a real unit can be physically present. Precedent exists: the scan-required rule is liftable by manager PIN (`SaleFlow.tsx:2098-2101`), and price overrides use `/api/pos/approve` (`RegisterShell.tsx:1173-1189`).

PART 2 — BUG B: the completed sale is resurrected by the next unlock

2.1 What is supposed to happen (and mostly does)

`onComplete` in `RegisterShell.tsx:1437-1454` is careful and correct:

```
onComplete={() => {
  setBanner(null);
  activeSaleRef.current = null;              // :1443
  try { posStorageRemove(ACTIVE_SALE_KEY); } // :1445
  catch {}
  setResumeSnapshot(null);                   // :1449
  setLoadedOrderId(null);                    // :1452
  lock();                                    // :1453
  void flush();
}}
```

It nulls the live ref, deletes the persisted snapshot, and clears the resume state **before** calling `lock()`. The comment at `:1441-1442` shows the author anticipated this exact hazard: "*a COMPLETED sale must never be re-parked by lock(); clear the live ref + stored snapshot first.*"

So why does it come back?

2.2 Failure 1 — the snapshot effect re-arms the ref after it is cleared

`SaleFlow.tsx:481-483`:

```
useEffect(() => {
  onSnapshotRef.current?.({ verdict, cart, medicalCard, member });
}, [verdict, cart, medicalCard, member]);
```

This pushes live sale state up to the shell, which stores it at `RegisterShell.tsx:1114-1119`:

```
onSnapshot={(state) => {
  activeSaleRef.current = state;   // :1118
}}
```

There is no completion guard on either side. The effect fires whenever `verdict`, `cart`, `medicalCard`, or `member` change — and critically:

SaleFlow never clears `cart` or `verdict` when a sale completes. I searched for it explicitly: the only `setMember(null)` calls are at `SaleFlow.tsx:4276` and `:4440` (manual "remove member" buttons), and there is **no** `setCart([])` and **no** `setVerdict(null)` anywhere in the file. Completion is only `setStep("done")` (`SaleFlow.tsx:817`).

So on the `done` screen (`SaleFlow.tsx:825-847`) the component is still mounted and still holding the finished customer's verdict and

cart in state.

2.3 Failure 2 — the 2-minute idle timer is live during the `done` screen

`RegisterShell.tsx:913-923`:

```
useEffect(() => {
  if (screen !== "home") return;       // :914
  touchIdle();                         // :915  → setTimeout(lock, IDLE_LOCK_MS)
  ...
}, [screen, touchIdle]);
```

`IDLE_LOCK_MS = 2 * 60 * 1000` (`:155`).

The guard is `screen !== "home"`. **While a sale is running, `screen` is still `"home"`** — the SaleFlow render branch is `if (saleActive && ...)` at `:1068`, which is a *different* variable. The only `setScreen` calls are `:443`, `:906`, `:994`, `:1026`, and none of them run when a sale starts. Verified by listing every one.

Therefore: the budtender finishes a sale, the celebratory `done` screen is up (`SaleFlow.tsx:825`), and they walk away without tapping **"Done — lock register"** (`:842`). Two minutes later the idle timer fires `lock()` (`:893-907`) — **not** `onComplete`. And `lock()` opens with:

```
const lock = useCallback(() => {
  parkActiveSale();          // :894  ← re-persists the COMPLETED sale
  activeSaleRef.current = null;
  ...
```

`parkActiveSale` (`:864-889`) reads `activeSaleRef.current` — which §2.2 established is still populated with the completed customer's verdict and cart — and writes it back to `ACTIVE_SALE_KEY` at `:879`.

The completed sale has now been resurrected into storage.

2.4 Failure 3 — unlock faithfully restores it, past the age gate

`RegisterShell.tsx:1035-1046`:

```
const parked = parseActiveSale(posStorageGet(ACTIVE_SALE_KEY));       // :1035
const decision = evaluateResume(parked, pacificDayKey(new Date()), Date.now());
if (decision.resume) {
  setResumeSnapshot(decision.snapshot);                                // :1038
  ...
  setSaleActive(true);                                                 // :1045
}
```

`evaluateResume` (`active-sale-resume-core.ts:243-288`) checks TTL, that age is still ≥ 21, that the ID has not expired, and that any medical card is still valid. **It has no concept of "this sale was already completed"** — it is checking whether the ID is still good, and the ID *is* still good. It correctly returns `resume: true`.

Then `RegisterShell.tsx:1088-1091` seeds SaleFlow:

```
initialCart={resumedCart ?? resumeCart ?? loadedCart ?? undefined}     // :1088
initialMember={resumeSnapshot?.member ?? loadedMember ?? undefined}    // :1089
initialVerdict={resumeSnapshot?.verdict ?? undefined}                  // :1090
```

and `SaleFlow.tsx:425` does the rest:

```
const [step, setStep] = useState<Step>(initialVerdict ? "cart" : "idgate");
```

A present verdict starts the sale at `"cart"`, not `"idgate"`. That is the skipped age gate, exactly as reported. Cart restored at `:466`, member at `:469`.

Chain complete and proven end to end: completion doesn't clear SaleFlow state → the idle timer still runs on the `done` screen → `lock()` re-parks the finished sale → unlock re-validates the (still-valid) ID and restores it → `initialVerdict` bypasses the gate.

2.5 A second, independent leak in the same area

`lock()` (`:893-907`) clears `resumeCart` (`:898`) and `resumeSnapshot` (`:899`) but **never clears `loadedCart` or `loadedMember`**. Those two are only ever set at `:264-265` (declaration) and read at `:1088-1089`; the full list of their setters is at `RegisterShell.tsx:1046-1047`, `:1478-1479`, `:1508-1509`, `:1733-1734`. Since `:1088` falls back `resumedCart ?? resumeCart ?? loadedCart`, a website-order cart loaded via the pickup queue (`:1733`) can survive a lock and reappear for the next customer even when the age-gate path behaves.

I am flagging this as a **related but separate** defect found during recon. It has the same symptom family and should be fixed in the same slice, but I have not reproduced it on the iPad — it is proven by code reading only, and I will say so plainly rather than claim Michael saw it.

PART 3 — ROADMAP FOR THE FIX SLICE (Slice 14)

Ordered so each step is independently testable. No step depends on the Socket AppKey, so this can proceed while we wait.

Step 1 — a pure, tested stock-ceiling core (no UI yet)

New `src/lib/pos/stock-ceiling-core.ts`, pure and self-tested in the house style (`ok(...)` assertions, matching `low-stock-core.ts`):

- `sellableCeiling(unitsLeft): number | null` — returns `null` (= unlimited / unknown) for `null`, `undefined`, non-integer, or negative input, mirroring the guards at `low-stock-core.ts:78` and `held-stock-core.ts:88`.
- `clampToStock(requested, unitsLeft): { quantity, clamped, ceiling }` — never throws, never returns more than the ceiling when one is known.
- Tests must include: unknown → unlimited; 0 → cannot add at all; exact-fit allowed; over-by-one clamped; corrupt values ignored; `MAX_LINE_QUANTITY` still respected as the outer bound.

Step 2 — enforce inside the two mutation functions

Change `addToCart` (`sale-flow-core.ts:201`) and `setCartQuantity` (`:214`) to consult the ceiling. Because §1.3 proved all seven entry points funnel through these two, this single change covers tile taps, both scan paths, the steppers, and the future Socket path.

The functions must return **why** they clamped, not silently cap — a silent cap would be its own bug (the budtender taps `+` and nothing happens, with no explanation). Existing callers pass a bare array today, so the return shape must stay backward compatible or every call site gets updated in the same commit.

Guard: the existing self-test at `sale-flow-core.ts:560` must keep passing.

Step 3 — make the block real at tender

Add a stock condition to `canTender` (`SaleFlow.tsx:2272-2273`), alongside `limits.blocked`. Render it with the same visual grammar as the limit meter (`:2663-2670`) and the High-THC notice (`:2638-2643`) so it reads as one consistent family of rules rather than a new dialect.

Step 4 — manager override

Reuse the existing `/api/pos/approve` path already wired at `RegisterShell.tsx:1173-1189`, following the scan-required precedent (`SaleFlow.tsx:2098-2101`). Scope the override to one sale, never persisted — matching how price overrides are described at `SaleFlow.tsx:490-492`.

Step 5 — kill the resurrection (Bug B), three defences

Fix all three links in the §2.4 chain, not just one, so no single regression can bring the bug back:

1. **SaleFlow stops reporting a completed sale.** Suppress the `onSnapshot` effect (`SaleFlow.tsx:481-483`) once `step === "done"`, and/or clear `cart` / `verdict` / `member` / `medicalCard` at completion (`:817`).
2. **`lock()` must not park a completed sale.** Give `RegisterShell` an explicit "this sale is finished" flag that `parkActiveSale` (`:864`) checks before writing at `:879`. Today it relies purely on `activeSaleRef` having been nulled, which §2.2 showed gets re-armed.
3. **`lock()` clears the loaded-order state too** — add `setLoadedCart(null)` and `setLoadedMember(null)` to `:893-907`, closing §2.5.

Step 6 — regression tests that fail against today's code

Non-negotiable per the standing rule; each must be demonstrated red before it goes green:

- Add 5 of a product with `unitsLeft: 3` → cart holds 3, not 5.
- `unitsLeft: 0` → the item cannot be added at all.
- `unitsLeft: null` (custom sale, `custom-sale-core.ts:146`) → unrestricted, no regression.
- `canTender` is false when any line exceeds its known ceiling.
- **Bug B end-to-end:** complete a sale → fire the idle lock → unlock → assert storage has no snapshot, cart is empty, and `step === "idgate"`.
- `lock()` clears `loadedCart` / `loadedMember`.

Step 7 — verify on the iPad

Build to Michael's device and have him reproduce both original reports. Code review is not proof; the counter is.

PART 4 — OPEN QUESTIONS I WILL NOT GUESS AT

1. **Should an out-of-stock item be hidden or greyed on the tile grid, or still tappable with a clear refusal?** I lean toward visible-but-refused (hiding products makes staff think the menu is broken), but this is the owner's call.
2. **Is the `oversold` report from `sale-decrement-core.ts:214` surfaced to anyone today?** Not yet traced. Stated as unknown rather than assumed.
3. **Who may override?** Manager PIN only, or any employee? Precedent (`RegisterShell.tsx:1173-1189`) is manager/lead, and I will follow that unless told otherwise.
4. **Do we block on the CACHED number, or force a menu refresh first?** Blocking on a stale cache could refuse a real sale. My recommendation is block + override + a visible "menu last refreshed" cue, but I want the owner's agreement before building it.

PART 5 — WHAT I DID NOT DO

- No code was changed. This is recon, as instructed.
- I did not run the app or reproduce either bug on hardware; every conclusion is from reading source. The Bug B chain is proven link by link, but the final confirmation is Michael's iPad.
- I did not inspect the `pos_orders` DB rows to see whether an oversold sale has already happened in production.